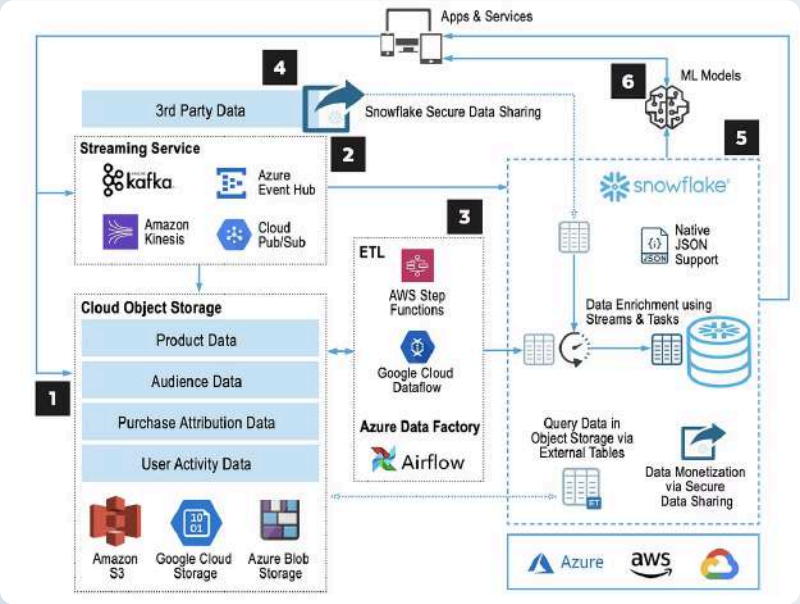



# Day 5: Model Deployment & Responsible AI

Azure Data Scientist Training · DP-100 Certification Prep · 4-Hour Session

DAY 5 OF 5

THEORY + HANDS-ON LABS

# Session at a Glance

| | |
|---|---|
| ⏱️ **Duration**<br>4 hours total | 📖 **Theory**<br>~1 hour · 25–30% |
| 🧪 **Labs**<br>~3 hours · 70–75% | 🎯 **Focus**<br>Deploy, Test & Explain |

# Day 5 Learning Objectives

1

### Model Deployment

Understand model registry, versioning, and lifecycle management in Azure ML

2

### REST API Endpoints

Expose trained models as REST APIs consumable by any application

3

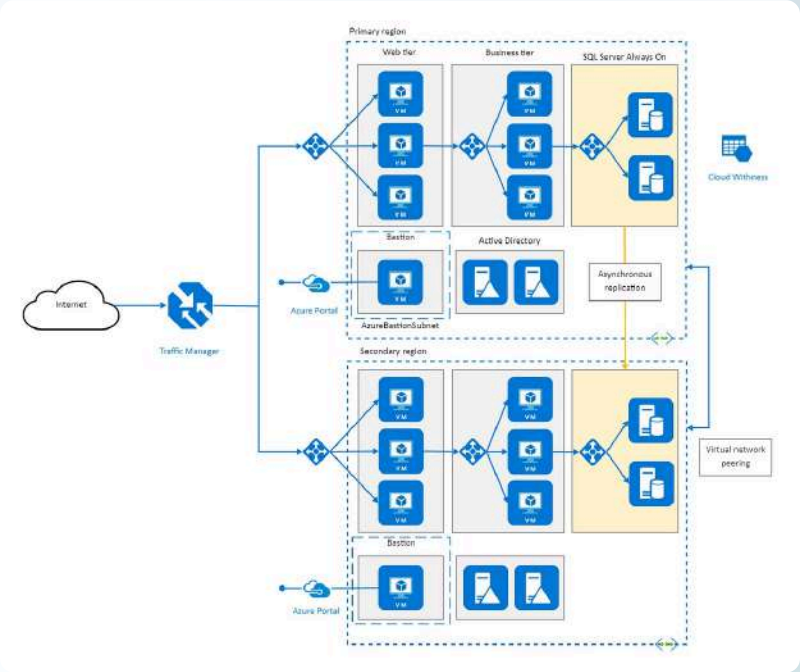
### Inference Types

Distinguish real-time vs. batch inference with real-world use cases

4

### Responsible AI

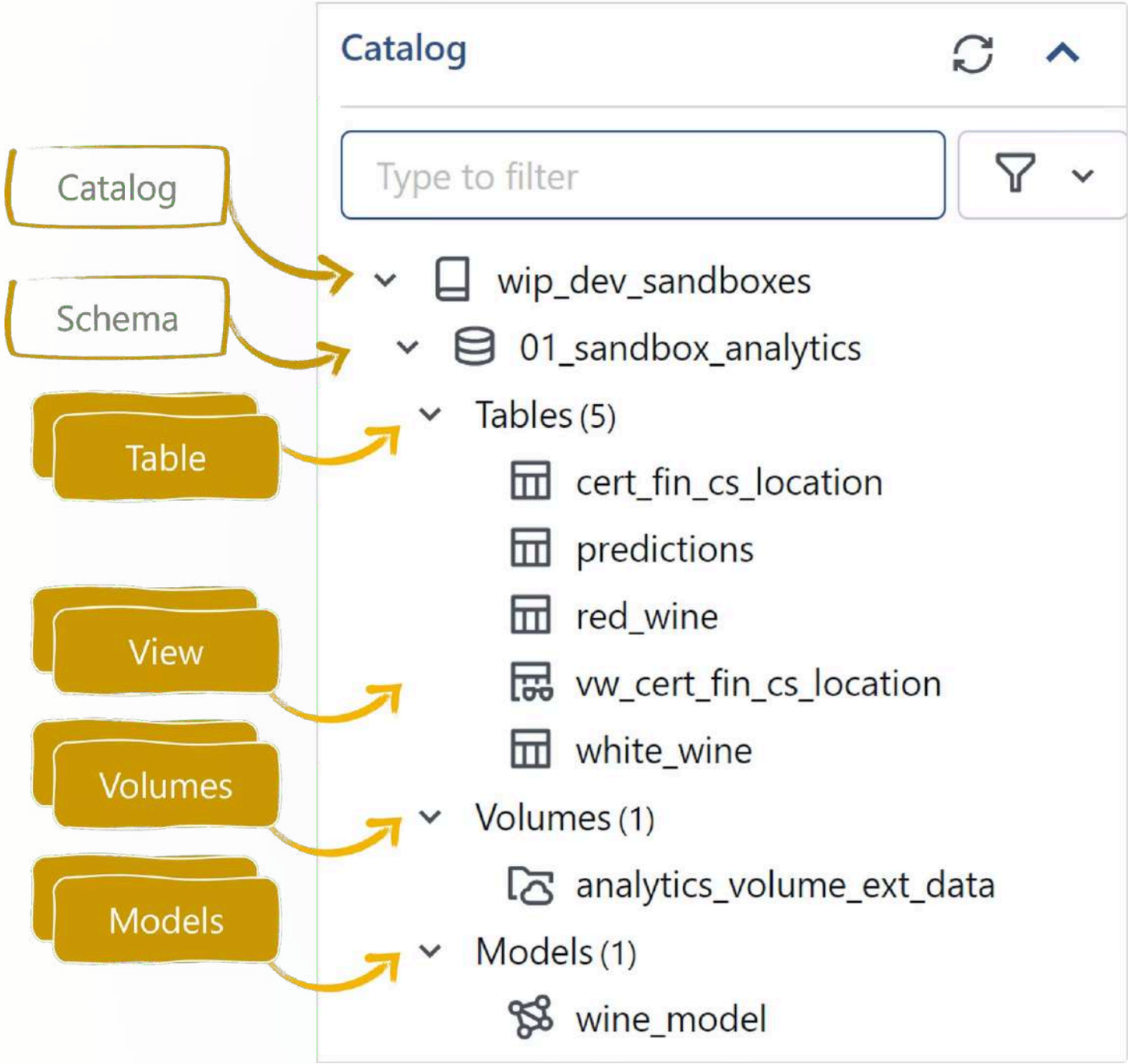
Apply bias detection, explainability, and fairness tools in Azure ML




CHAPTER 1

# Model Deployment in Azure ML

# What Is the Model Registry?



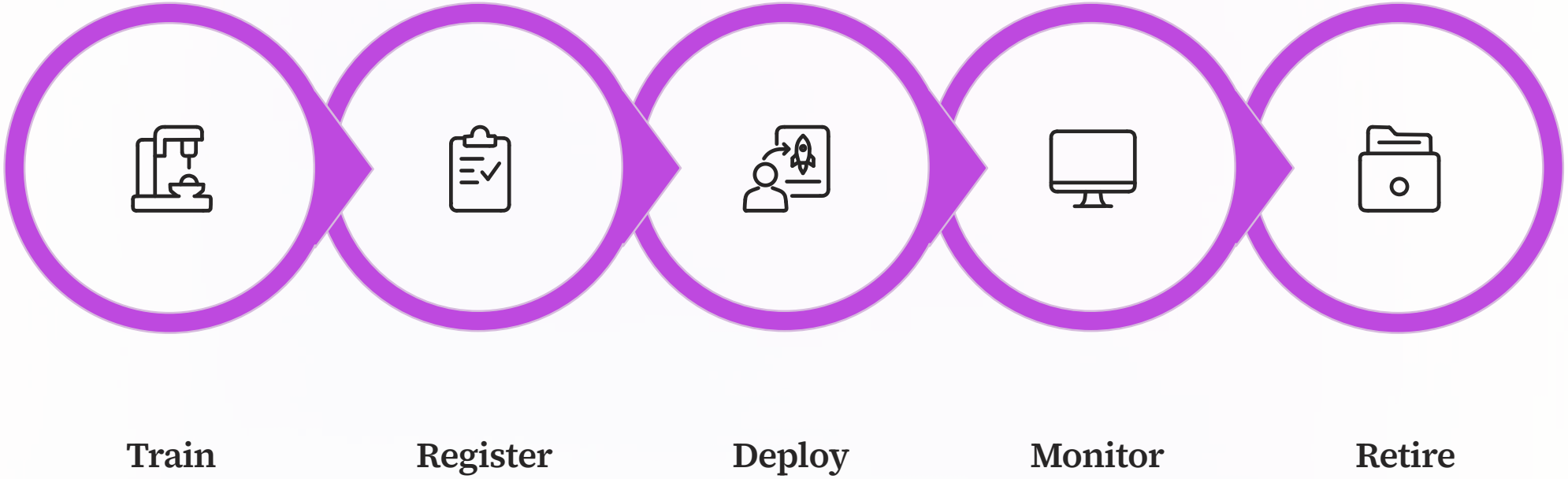

The **Azure ML Model Registry** is a centralized repository where trained models are stored, versioned, and managed throughout their lifecycle.

→ **Store**
Persist trained model artifacts (weights, parameters, metadata)

→ **Version**
Each registration creates a new version — full history preserved

→ **Deploy**
Reference registered models directly in deployment configurations

# Model Lifecycle in Azure ML




Every model progresses through a predictable lifecycle. The registry acts as the single source of truth, ensuring teams can trace, compare, and roll back any model version.

# Why Model Versioning Matters

### 🔁 Reproducibility

Re-create any prior experiment result with a specific version

### 🔄 Rollback

Instantly revert to a prior version if a new deployment underperforms

### 📊 Comparison

A/B test two model versions serving the same endpoint

### ✅ Governance

Audit trails for compliance — who registered what, and when

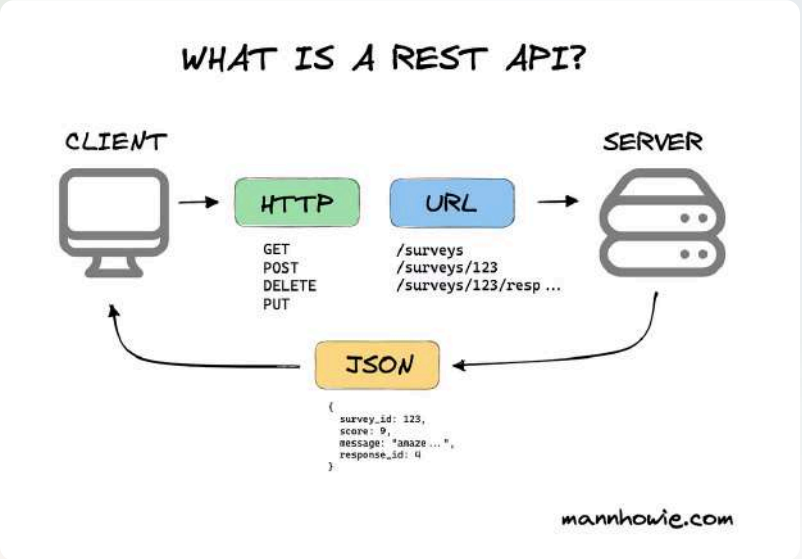
# Registering a Model — Python SDK

Use the Azure ML SDK to register a local model file or a model produced by a completed training run:

```
from azureml.core import Model, Workspace

ws = Workspace.from_config()

model = Model.register(
 workspace=ws,
 model_path="outputs/model.pkl", # local file or run output
 model_name="credit-risk-model",
 tags={"framework": "sklearn", "accuracy": "0.92"},
 description="Credit risk classifier v1"
)

print(f"Registered: {model.name} v{model.version}")
```

CHAPTER 2

# Inference Endpoints & REST APIs



# Models as REST APIs

## How It Works

Once deployed, a model becomes a **web service** that accepts HTTP POST requests and returns JSON predictions. Any client — web app, mobile app, enterprise system — can integrate without knowing the underlying model.

- Input: JSON payload with feature values
- Output: JSON response with predictions or scores
- Auth: API keys or Azure AD tokens

### Client App

Sends POST request with input data

### REST Endpoint

Hosted scoring container receives request

### Prediction

Returns JSON response to caller

# Real-Time vs. Batch Inference

## Real-Time Inference

**Low-latency, on-demand predictions.** The model responds within milliseconds to individual requests.

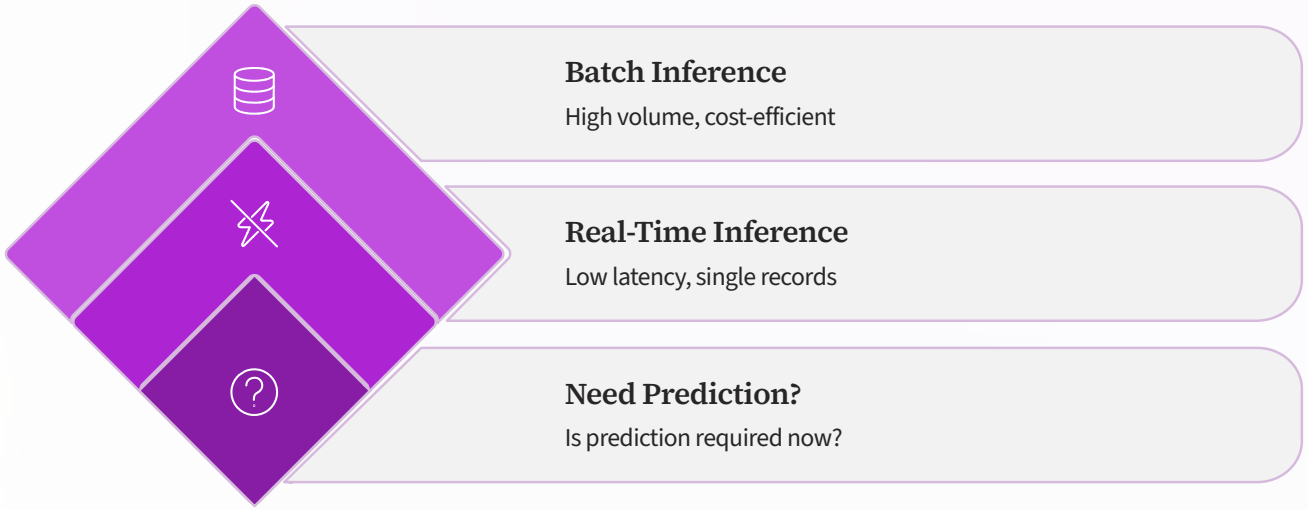
- Fraud detection during a transaction
- Product recommendation at checkout
- Real-time credit scoring

## Batch Inference

**High-throughput, scheduled predictions.** The model scores large datasets in bulk at defined intervals.

- Monthly demand forecasting
- Nightly churn risk scoring
- Weekly inventory predictions

# Choosing the Right Inference Mode

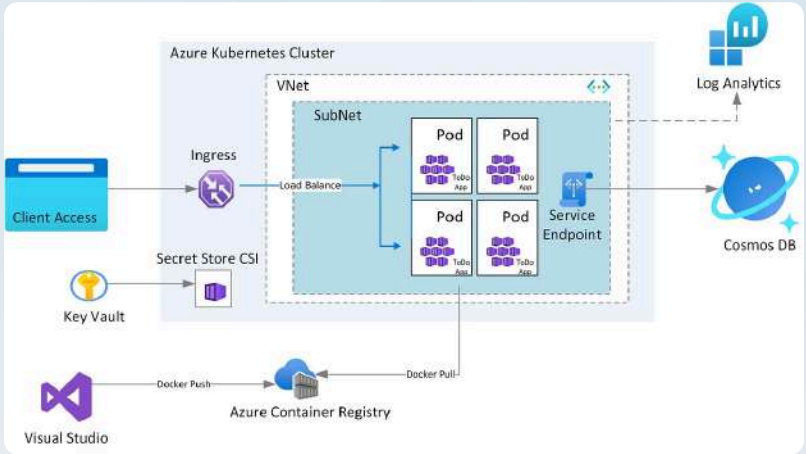



## Key Considerations

- **Latency requirement:** milliseconds → real-time; minutes/hours → batch
- **Volume:** individual records → real-time; millions of rows → batch
- **Cost:** batch is significantly cheaper for large workloads
- **Frequency:** continuous → real-time; scheduled → batch

CHAPTER 3

# Azure ML Deployment Options



# Deployment Targets Compared

### Azure Container Instances (ACI)

**Best for:** Development, testing, and low-traffic proof-of-concept deployments

- Fastest to provision — no cluster required
- Low cost for short-lived workloads
- Limited scalability and no auto-scaling

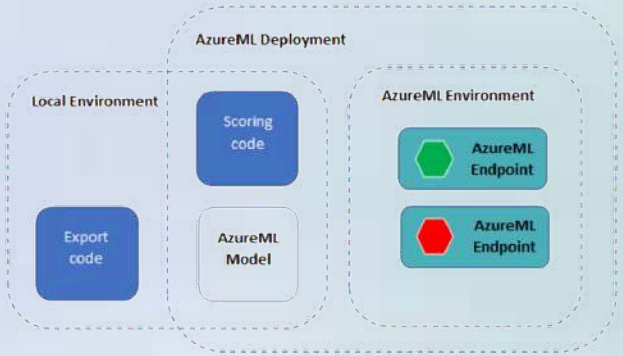
### Azure Kubernetes Service (AKS)

**Best for:** Production workloads requiring scalability, reliability, and SLA

- Auto-scales replicas under load
- Supports GPU inference and custom networking
- Requires cluster setup and management overhead

# ACI vs. AKS at a Glance

| Dimension | ACI | AKS |
|---|---|---|
| Setup Time | Minutes | 10–20 minutes |
| Auto-Scaling | No | Yes |
| Cost | Low / Pay-per-use | Higher (cluster overhead) |
| SLA | No SLA | 99.9% uptime SLA |
| GPU Support | Limited | Full support |
| Recommended Use | Dev / Test | Production |




CHAPTER 4

# Core Deployment Components

# The Three Deployment Pillars

### Scoring Script

A Python file (`score.py`) with two functions: `init()` loads the model into memory at startup, and `run()` accepts input data and returns predictions on every request.

### Environment Configuration

Defines the **Python version**, required packages (scikit-learn, pandas, etc.), and Docker base image. Ensures the container runs with identical dependencies every time.

### Endpoint

The deployed web service that exposes the model. Holds the REST URL, authentication keys, and routes traffic to the underlying scoring container.

# Anatomy of a Scoring Script

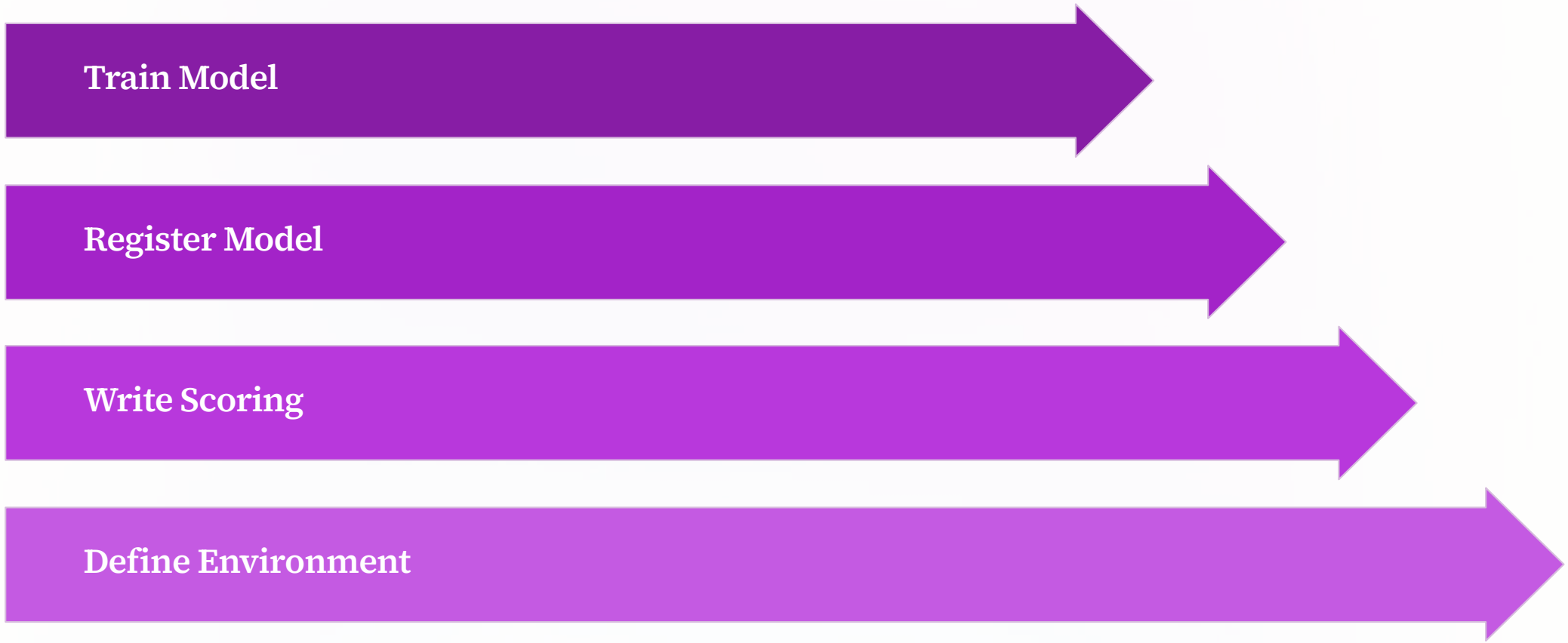
```
import joblib
import json
import numpy as np
from azureml.core.model import Model

def init():
    global model
    model_path = Model.get_model_path("credit-risk-model")
    model = joblib.load(model_path)

def run(raw_data):
    try:
        data = json.loads(raw_data)["data"]
        result = model.predict(np.array(data))
        return result.tolist()
    except Exception as e:
        return {"error": str(e)}
```

`init()` runs once at container startup. `run()` is called on every incoming prediction request.

# Defining the Inference Environment

```
from azureml.core import Environment
from azureml.core.conda_dependencies import CondaDependencies

env = Environment("scoring-env")
env.python.conda_dependencies = CondaDependencies.create(
 python_version="3.8",
 pip_packages=[
 "scikit-learn==1.0.2",
 "joblib",
 "numpy",
 "pandas",
 "azureml-defaults"
 ]
)

# Pin to a specific base image for reproducibility
env.docker.base_image = "mcr.microsoft.com/azureml/openmpi4.1.0-ubuntu20.04"
```

# End-to-End Deployment Workflow




Each step is a discrete, repeatable action. Teams can automate this entire pipeline using Azure ML Pipelines or GitHub Actions for CI/CD model deployment.

# Deploying to Azure Container Instances

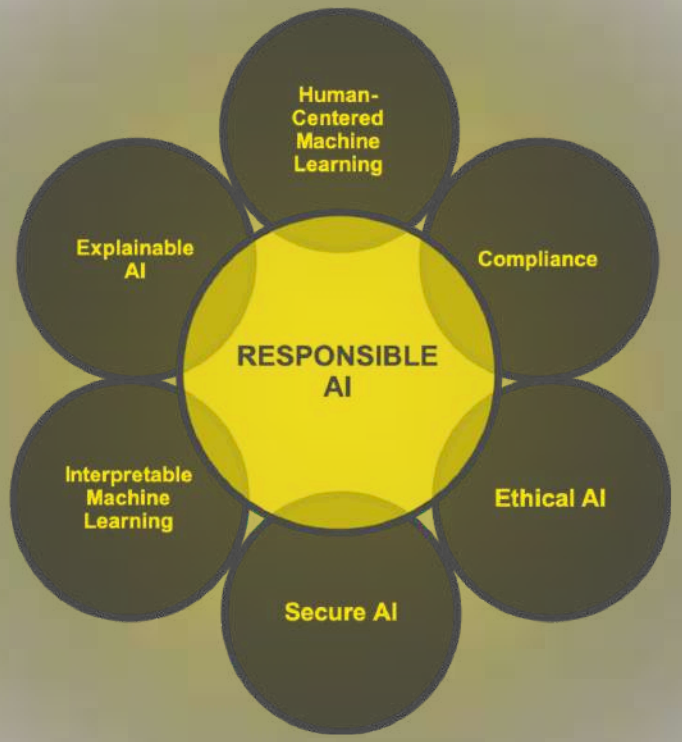
```
from azureml.core.webservice import AciWebservice, Webservice
from azureml.core.model import InferenceConfig

inference_config = InferenceConfig(
 entry_script="score.py",
 environment=env
)

aci_config = AciWebservice.deploy_configuration(
 cpu_cores=1,
 memory_gb=1,
 auth_enabled=True
)

service = Model.deploy(
 workspace=ws,
 name="credit-risk-endpoint",
 models=[model],
 inference_config=inference_config,
 deployment_config=aci_config
)
service.wait_for_deployment(show_output=True)
```

# Consuming the Deployed API

```
import requests
import json

# Retrieve endpoint URL and key
scoring_uri = service.scoring_uri
api_key = service.get_keys()[0]

# Prepare sample input
input_data = json.dumps({
 "data": [[45, 2, 30000, 1, 0, 1, 3, 500, 2]]
})

headers = {
 "Content-Type": "application/json",
 "Authorization": f"Bearer {api_key}"
}

response = requests.post(scoring_uri, data=input_data, headers=headers)
print("Prediction:", response.json())
```

CHAPTER 5

# Responsible AI in Azure Machine Learning



# Why Responsible AI Matters

Modern ML models influence high-stakes decisions. When models are biased or opaque, real people are harmed.

### Banking

Biased loan models may deny credit unfairly based on demographics

### Healthcare

Diagnostic models can underperform for under-represented patient groups

### Hiring

Resume screening models can perpetuate historical hiring biases

### Insurance

Risk scoring models may penalize certain groups without justified cause

# Microsoft's Responsible AI Principles

### Fairness

Equal treatment across demographic groups

### Accountability

Human oversight and auditability of AI systems

### Transparency

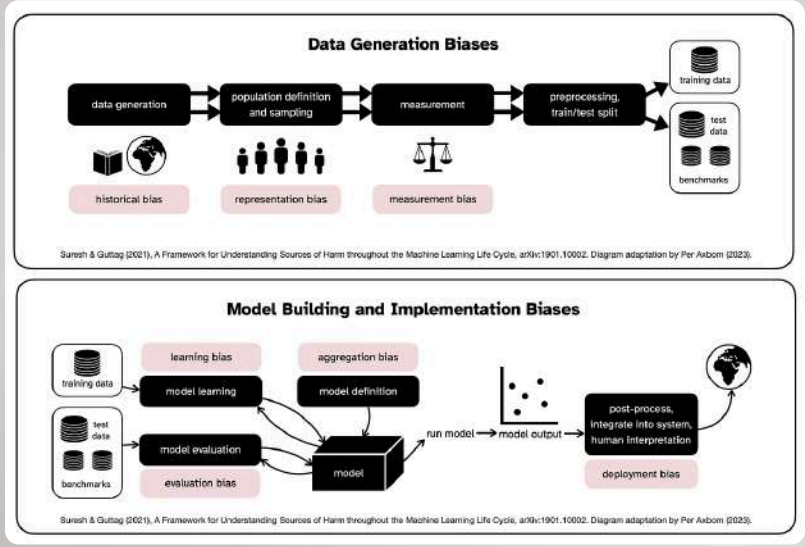
Understandable model decisions and processes

### Reliability

Consistent, safe behavior under real-world conditions

### Privacy

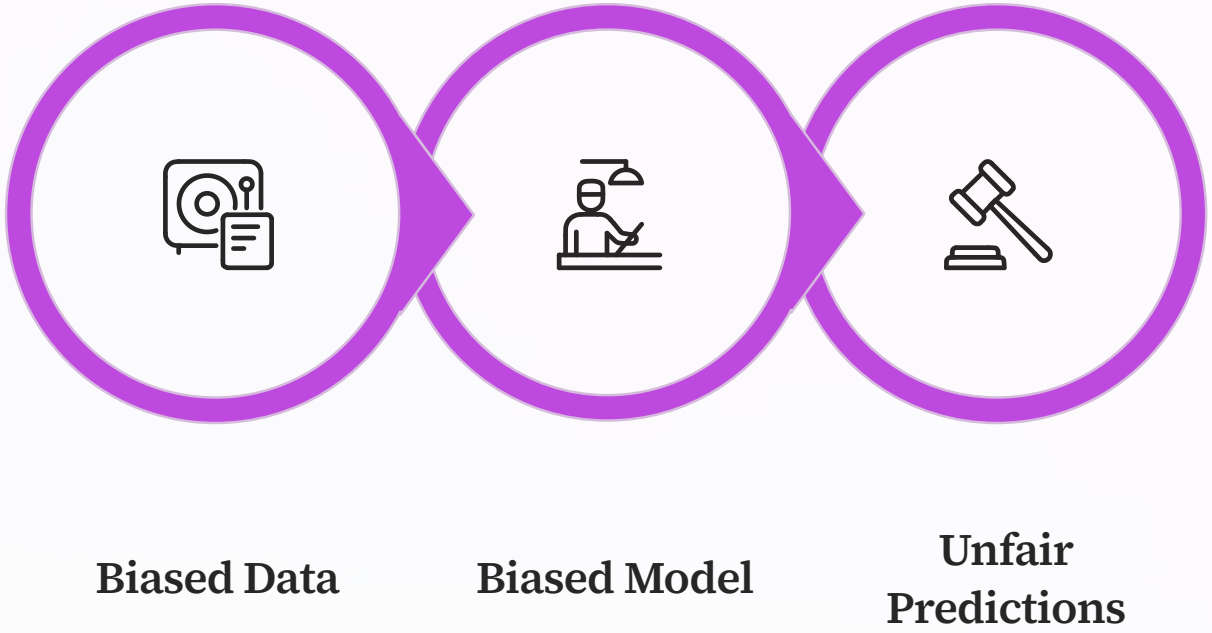
Protect personal data throughout the ML lifecycle




CHAPTER 6

# Bias Detection

# How Bias Enters ML Models




Bias is rarely intentional. It enters through historical inequalities baked into data, unrepresentative sampling, or proxy variables that correlate with protected attributes like race or gender.

# Types of Bias to Watch For

### Historical Bias

Training data reflects past societal inequalities. A hiring model trained on historical decisions perpetuates those patterns, even without explicit demographic features.

### Sampling Bias

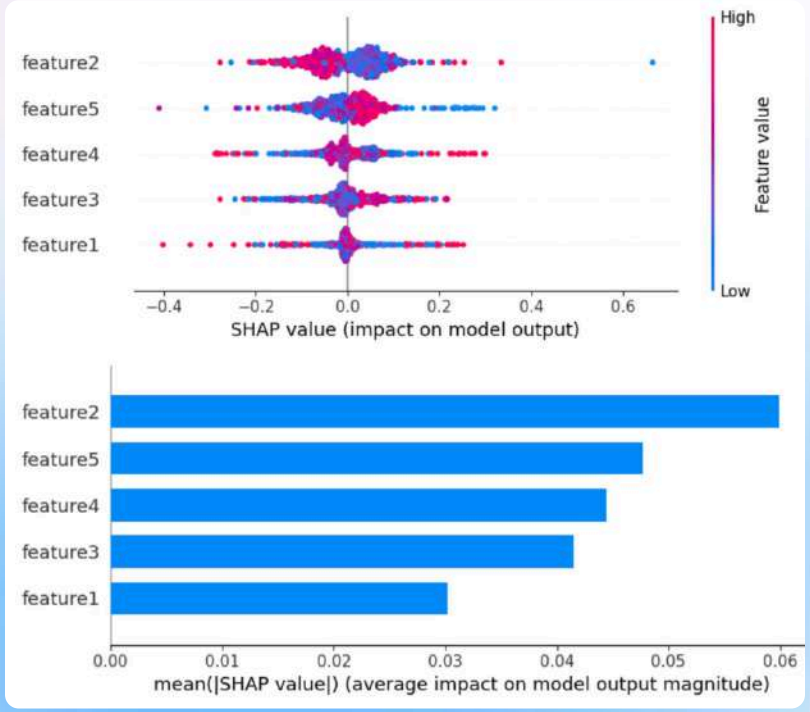
Under-represented groups in training data lead to poor model performance for those groups. Rare events and minority classes are especially vulnerable.

### Measurement Bias

Systematic errors in how data is collected for certain groups. For example, different diagnostic thresholds applied across demographic groups.

CHAPTER 7

# Model Explainability



# What Is Model Explainability?

Explainability tools help data scientists and stakeholders **understand why** a model made a specific prediction — not just what it predicted.

- **Global explanations:** Which features matter most across all predictions?
- **Local explanations:** Why did the model predict X for this specific record?
- **Trust building:** Regulators and business users can verify logic
- **Debugging:** Spot spurious correlations before deployment

### SHAP

SHapley Additive exPlanations — game-theory-based feature attribution

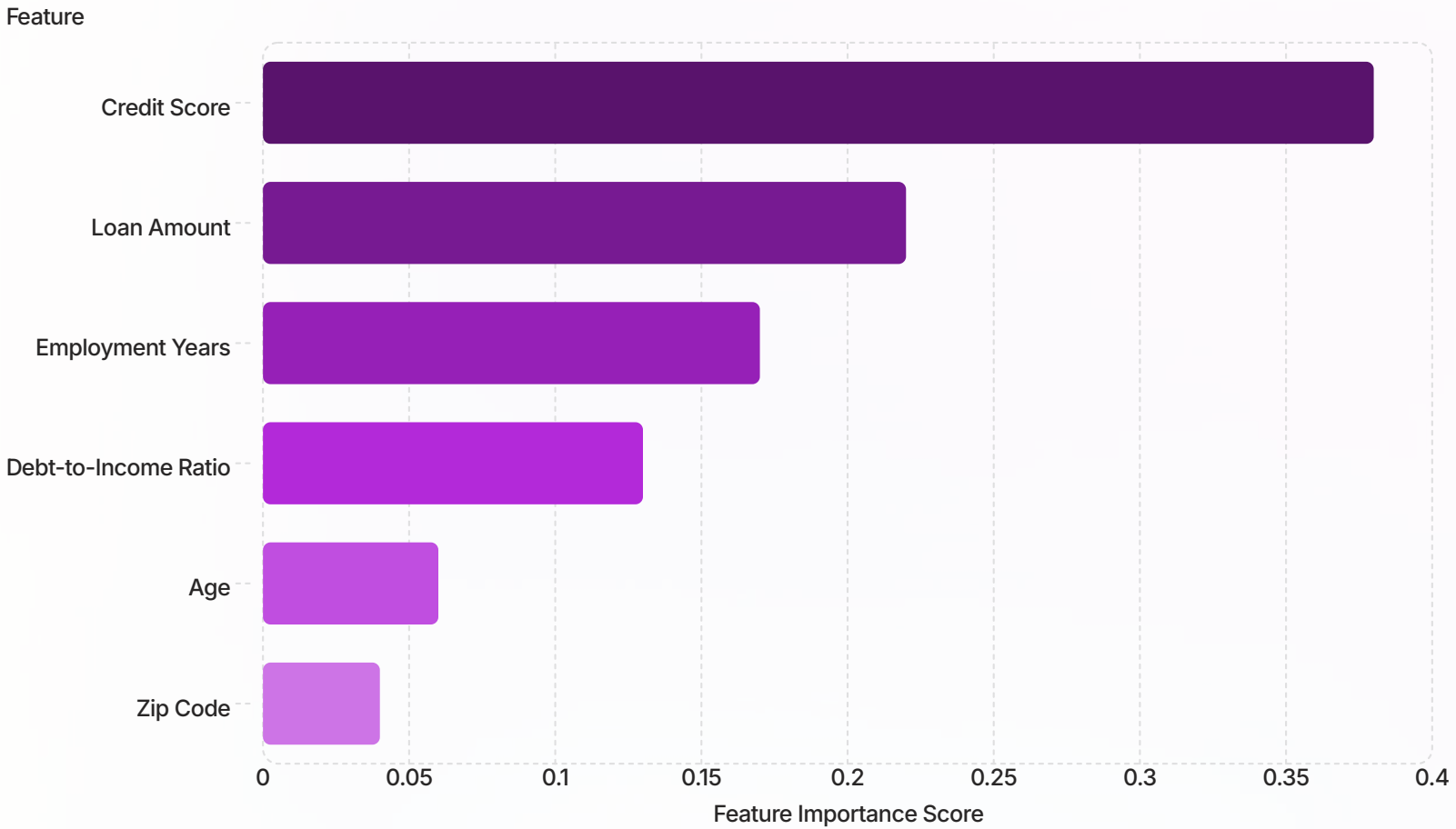
### LIME

Local Interpretable Model-agnostic Explanations — local approximation

### InterpretML

Microsoft open-source toolkit integrated with Azure ML

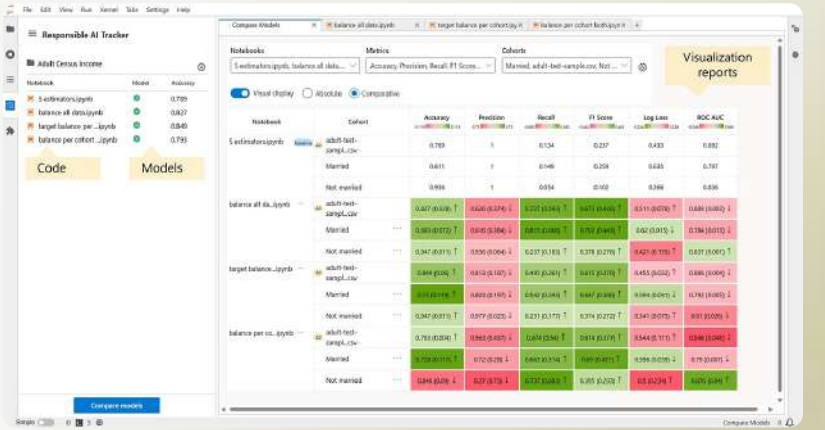
# Feature Importance — Practical Example




High importance for "Zip Code" — a geographic proxy — may signal latent demographic bias. Explainability exposes these patterns before production.

# The Responsible AI Dashboard

The **Azure ML Responsible AI Dashboard** is a unified interface that consolidates error analysis, model explainability, fairness assessment, and causal analysis into a single workflow — available directly in Azure ML Studio.



# Responsible AI Dashboard — Key Tools

## Error Analysis

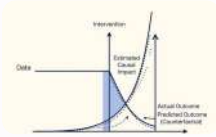
Identifies cohorts of data where the model fails most. Pinpoints blind spots using a decision tree visualizer and heat map.

## Model Explanations

Global and local feature importance using SHAP. Shows how each feature pushes predictions up or down for any individual record.

## Fairness Assessment

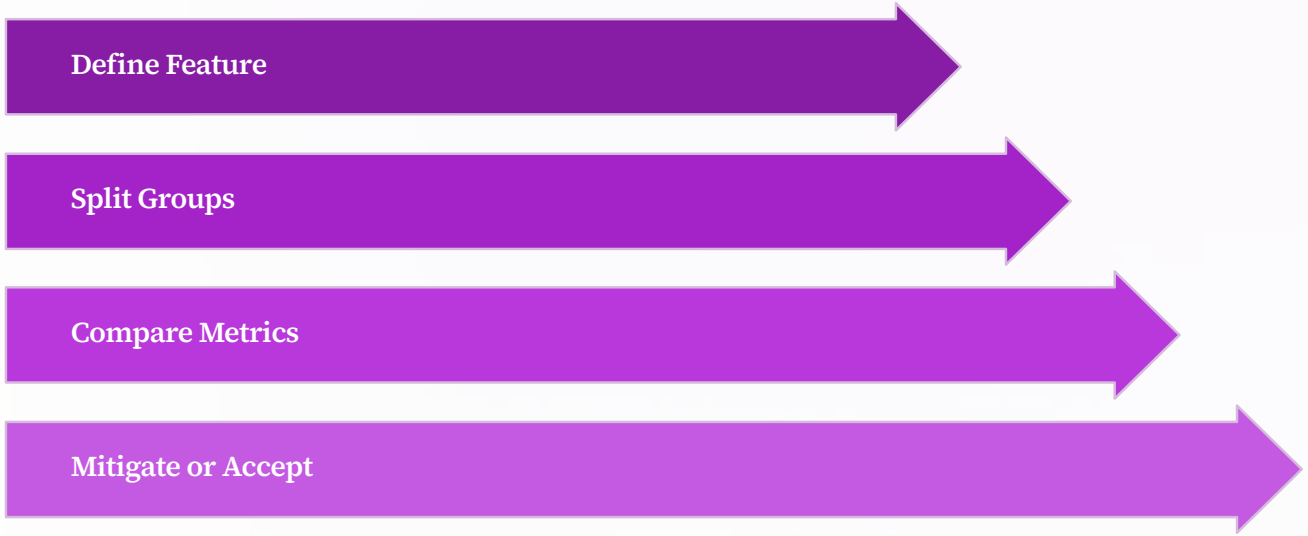
Compares model performance metrics across protected groups (gender, race, age). Flags disparate impact before deployment.



## Causal Analysis

Estimates causal effects of features on outcomes. Helps distinguish genuine causal drivers from spurious correlations.

# Fairness Metrics in Azure ML




## Key Fairness Metrics

- **Demographic Parity:** Equal positive prediction rates across groups
- **Equalized Odds:** Equal TPR and FPR across groups
- **Equal Opportunity:** Equal true positive rate for all groups
- **Disparate Impact Ratio:** Ratio of favorable outcomes between groups

# Enabling Responsible AI Dashboard — SDK

```
from raiwidgets import ResponsibleAIDashboard
from responsibleai import RAIInsights

rai_insights = RAIInsights(
 model=model,
 train=train_df,
 test=test_df,
 target_column="default",
 task_type="classification"
)

# Add components
rai_insights.explainer.add()
rai_insights.error_analysis.add()
rai_insights.fairness.add(
 sensitive_features=["gender", "age_group"],
 fairness_metrics=["demographic_parity_difference"]
)

rai_insights.compute()
ResponsibleAIDashboard(rai_insights)
```

# Day 5 Hands-On Labs

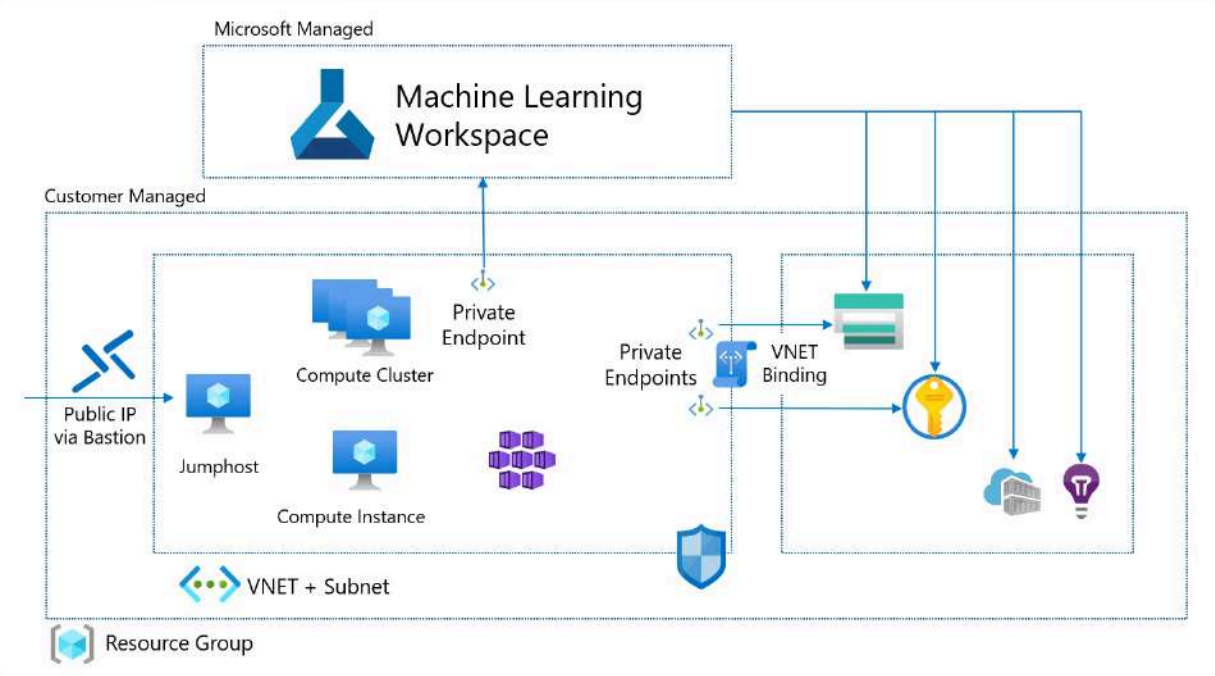
Four guided labs putting today's theory into practice — from model registration through explainability analysis.

LABS 13–16

~3 HOURS

LAB 13

# Register Model in Azure ML




## Objective

Save a trained model and register it in the Azure ML Model Registry with tags, descriptions, and versioning.

## Key Steps

1. Load or train a scikit-learn classifier
2. Save model artifact using `joblib.dump()`
3. Register with `Model.register()`
4. Verify version in Azure ML Studio UI

Duration: ~30 minutes

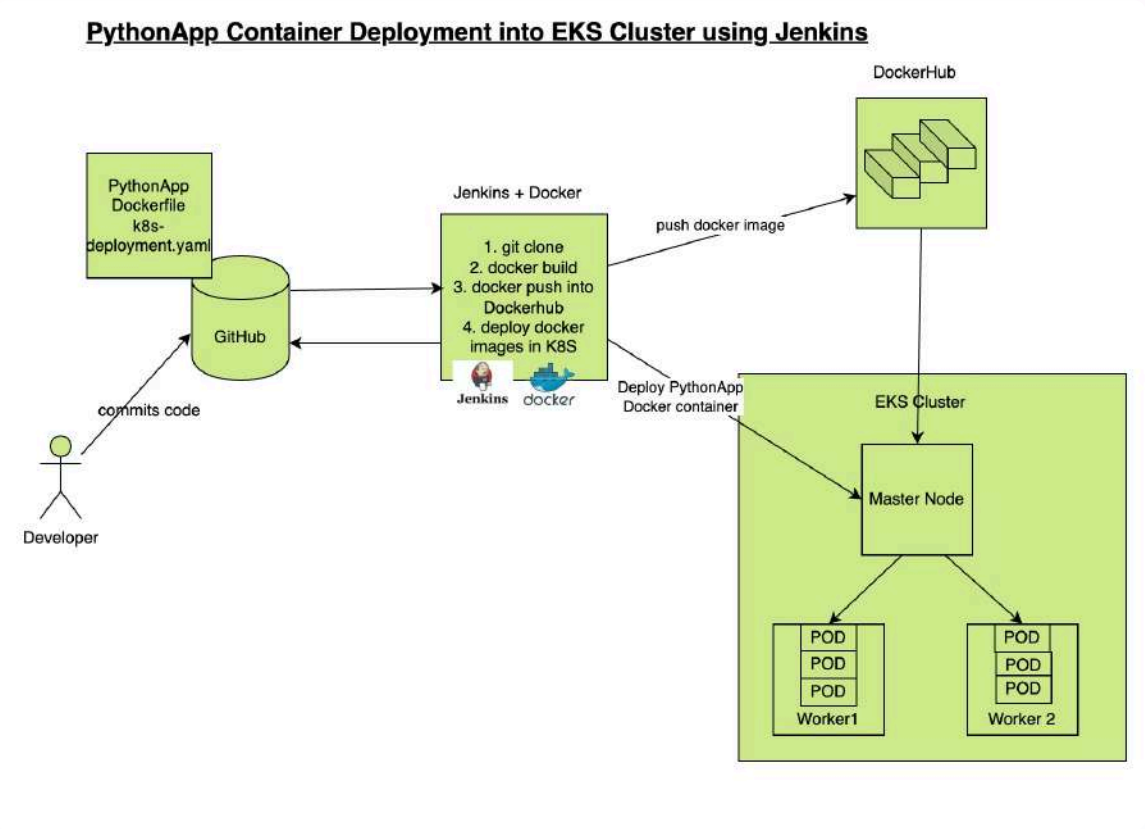
LAB 13

# Lab 13 — Code Walkthrough

```
from azureml.core import Workspace, Model
import joblib

ws = Workspace.from_config()

# Save model locally first
joblib.dump(trained_model, "outputs/model.pkl")

# Register in model registry
registered_model = Model.register(
 workspace=ws,
 model_path="outputs/model.pkl",
 model_name="credit-risk-model",
 tags={
 "algorithm": "RandomForest",
 "accuracy": "0.92",
 "dataset": "lending-club-2024"
 },
 description="Binary classifier for credit default risk"
)

print(f"
```

LAB 14

# Deploy Model as REST API




## Objective

Create a scoring script, define the inference environment, and deploy the registered model to an Azure Container Instances endpoint.

## Key Steps

1. Write `score.py` with `init()` and `run()`
2. Define `InferenceConfig` with environment
3. Configure `AciWebservice` deployment settings
4. Deploy and wait for healthy state

Duration: ~45 minutes

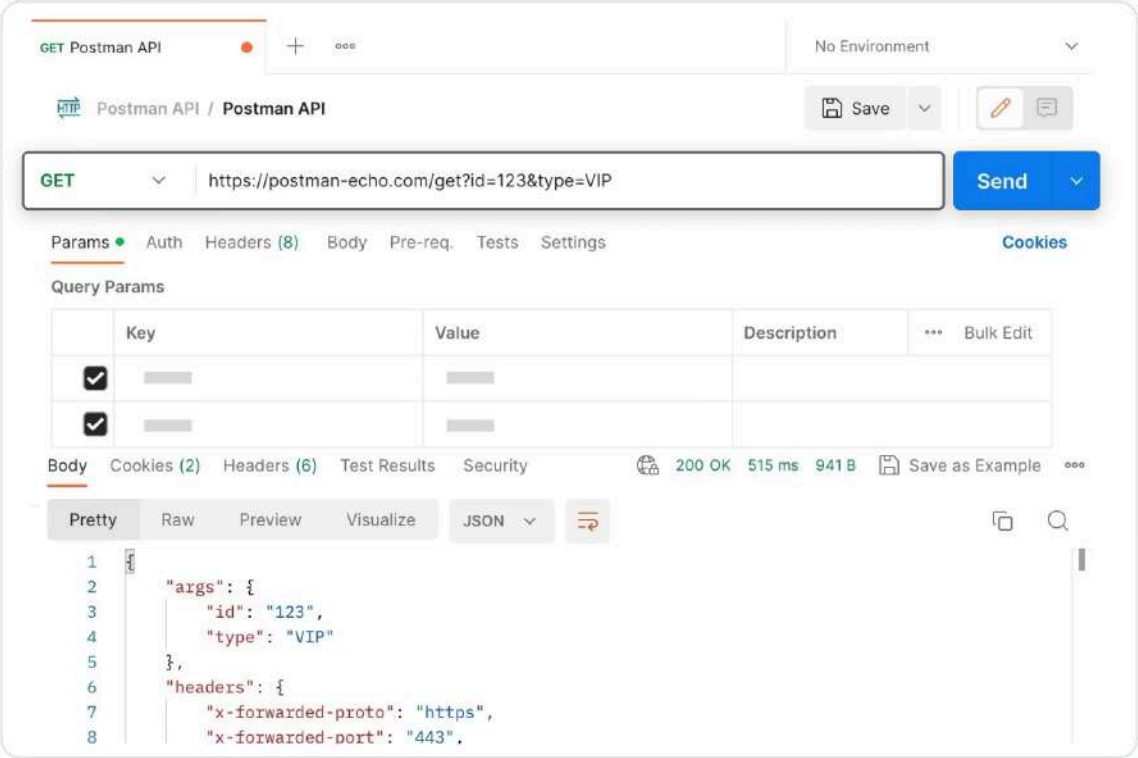
LAB 14

# Lab 14 — Deployment Code

```
from azureml.core.webservice import AciWebservice
from azureml.core.model import InferenceConfig, Model
from azureml.core import Environment

# Build environment
env = Environment.from_conda_specification(
 name="scoring-env",
 file_path="environment.yml"
)

inference_config = InferenceConfig(
 entry_script="score.py",
 environment=env
)

aci_config = AciWebservice.deploy_configuration(
 cpu_cores=1, memory_gb=1,
 auth_enabled=True,
 tags={"lab": "14", "stage": "dev"}
)

model = Model(ws, name="credit-risk-model")
service = Model.deploy(ws, "credit-risk-svc", [model],
 inference_config, aci_config)
service.wait_for_deployment(show_output=True)
print("Endpoint:", service.scoring_uri)
```

LAB 15

# Test Model Endpoint




## Objective

Send HTTP POST requests to the deployed endpoint using Python and validate that predictions are accurate and the service is healthy.

## Key Steps

1. Retrieve scoring URI and API key from service
2. Construct JSON input payload
3. Send request using `requests.post()`
4. Validate prediction output and response codes

Duration: ~30 minutes

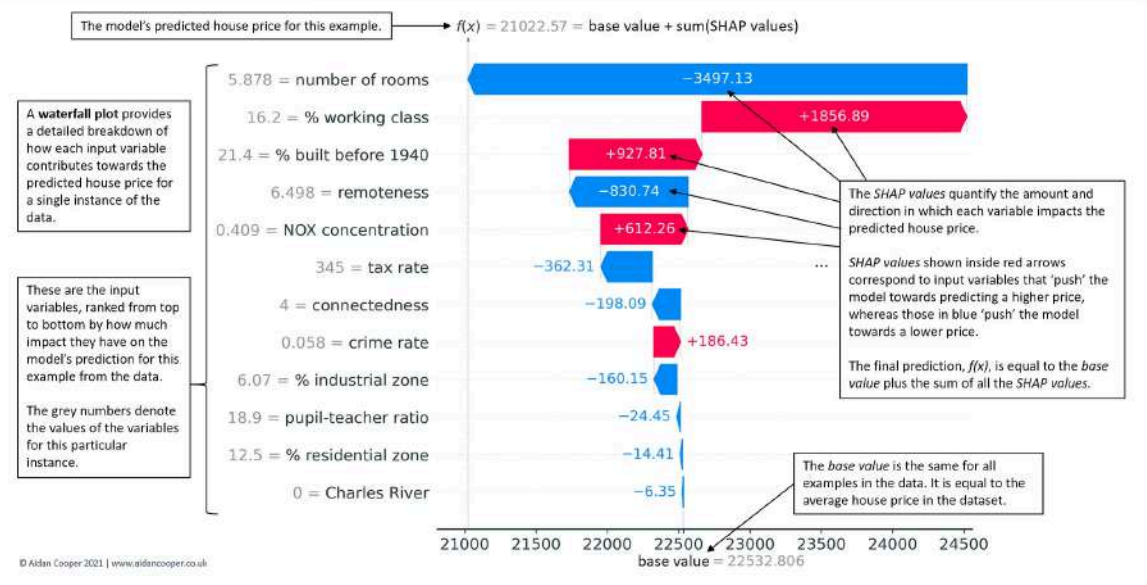
LAB 15

# Lab 15 — Endpoint Testing Code

```
import requests
import json

scoring_uri = service.scoring_uri
api_key = service.get_keys()[0]

# Sample: 3 customers with different risk profiles
test_cases = {
 "data": [
 [720, 1, 25000, 5, 0, 1, 2, 800, 3], # Low risk
 [580, 3, 45000, 2, 1, 0, 4, 200, 8], # High risk
 [650, 2, 30000, 4, 0, 1, 3, 500, 5] # Medium risk
 ]
}

headers = {
 "Content-Type": "application/json",
 "Authorization": f"Bearer {api_key}"
}

resp = requests.post(scoring_uri, json.dumps(test_cases), headers=headers)
print("Status:", resp.status_code)
print("Predictions:", resp.json())
```

LAB 16

# Model Explainability Lab




## Objective

Use Azure ML interpretability tools to generate feature importance scores and analyze which features drive model predictions.

## Key Steps

1. Install `interpret-community` package
2. Create a `TabularExplainer` from the trained model
3. Generate global and local explanations
4. Visualize results in the ExplanationDashboard

Duration: ~45 minutes

LAB 16

# Lab 16 — Explainability Code

```
from interpret.ext.blackbox import TabularExplainer
from raiwidgets import ExplanationDashboard
import pandas as pd

# Initialize explainer with background data
explainer = TabularExplainer(
 model=trained_model,
 initialization_examples=X_train,
 features=feature_names,
 classes=["No Default", "Default"]
)

# Global explanations — all predictions
global_exp = explainer.explain_global(X_test)
print("Top features:", global_exp.get_feature_importance_dict())

# Local explanation — single record
local_exp = explainer.explain_local(X_test[:5])

# Launch interactive dashboard
ExplanationDashboard(global_exp, trained_model, dataset=X_test)
```

# Labs Summary — What You Built

### Lab 13 — Register Model

Stored and versioned a trained model in the Azure ML Model Registry

### Lab 14 — Deploy REST API

Deployed the model as an ACI-hosted web service with auth-enabled endpoint

### Lab 15 — Test Endpoint

Sent live HTTP requests and validated prediction responses end-to-end

### Lab 16 — Explainability

Generated global and local feature importance and visualized model logic

# Cost Optimization Tips for Labs

1

### Use ACI, Not AKS

ACI is significantly cheaper for lab work — no idle cluster charges

2

### Limit CPU & Memory

Set `cpu_cores=1` and `memory_gb=1` — enough for lab workloads

3

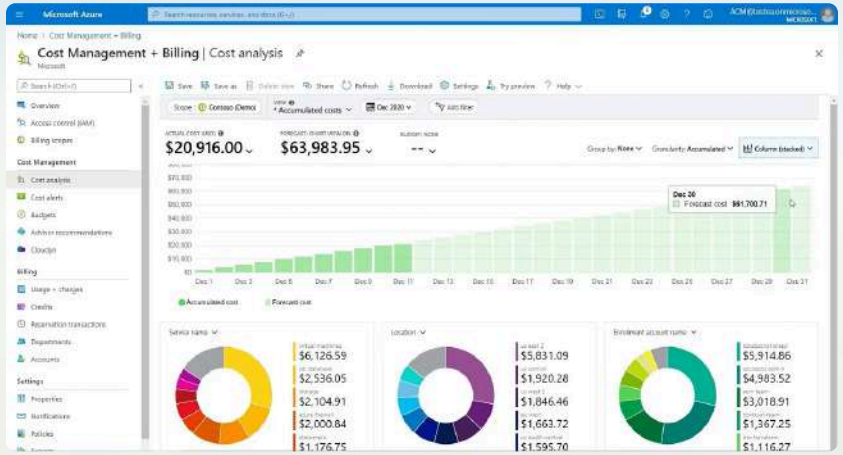
### Delete Endpoints

Run `service.delete()` immediately after each lab to stop billing

4

### Use Free Tier Compute

Azure ML free compute cluster tier covers basic inference workloads



# Cleaning Up After Labs

```
# Delete web service to stop charges immediately
service.delete()
print("
```

# DP-100 Exam Coverage — Day 5 Topics

20%

### Deploy Models

Register, configure, and deploy models to managed endpoints

15%

### Inference Endpoints

Real-time and batch inference patterns and configuration

10%

### Responsible AI

Fairness, explainability, and ethical AI tools in Azure ML

55%

### Other Topics

Days 1–4: Data prep, training, pipelines, AutoML

# Common Pitfalls to Avoid

### 🚫 Mismatched Dependencies

The scoring environment must use the **exact same package versions** as training. A version mismatch between training and inference is the #1 cause of deployment failures.

### 🚫 Leaving Endpoints Running

ACI endpoints accrue charges even with no traffic. Always delete test endpoints. Use `service.get_logs()` to diagnose issues before deleting.

### 🚫 Skipping Fairness Checks

Deploying without evaluating fairness metrics can expose your organization to regulatory risk. Run fairness analysis as part of every deployment checklist.

# Day 5 Key Takeaways

01

### Model Registry

Centralized versioning and lifecycle management for all trained models

02

### REST API Endpoints

ACI for dev/test; AKS for production — always auth-enabled

03

### Inference Modes

Real-time for low-latency decisions; batch for high-volume scheduled jobs

04

### Responsible AI

Bias detection, fairness metrics, and explainability are non-negotiable

05

### Azure ML Dashboard

Unified RAI tooling: error analysis, SHAP explanations, fairness evaluation